

Class-2

Topics: arrays, simple array algorithms, sorting (the patterns that matter), binary search (two templates), lower/upper bound and the query table, "find a repeated element" (four solutions), and run-length encoding revisited.

1. Arrays

An array is a block of **contiguous memory** holding elements of the same type.

Properties worth knowing cold:

- **Index access is $O(1)$** — $\text{address} = \text{base} + \text{index} \times \text{element_size}$. This is *the* superpower of arrays.
- **Contiguous memory = cache friendly**. Scanning an array is much faster in practice than scanning a linked list of the same size, even though both are " $O(n)$ ".
- **Insert/delete in the middle is $O(n)$** — everything after the position must shift.
- **Append at the end** is $O(1)$ amortized for dynamic arrays (Python `list`, Java `ArrayList`) — occasionally the array grows and copies everything.
- Static arrays have fixed size; dynamic arrays resize themselves.

Interview point of view — most array questions are really about one of these patterns:

1. **Two pointers** (from both ends, or slow/fast)
2. **Sliding window**
3. **Prefix sums**
4. **Sort first, then think**
5. **Binary search** (on the array, or on the *answer*)
6. **Use the index itself as information** — today's duplicate problem!
7. **In-place tricks** — yesterday's run-length encoding!

If you are stuck on an array problem, walk this list.

2. Warm-up algorithms

Linear search — $O(n)$

```
python
def linear_search(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i
    return -1
```

Min and max — $O(n)$, one pass

python

```
def find_min_max(a):
    mn = mx = a[0]
    for x in a[1:]:
        if x < mn: mn = x
        if x > mx: mx = x
    return mn, mx
```

Second max (and second min) — one pass, be careful!

python

```
def second_max(a):
    first = second = float('-inf')
    for x in a:
        if x > first:
            second, first = first, x      # old champion becomes runner-up
        elif first > x > second:
            second = x                    # strictly between them
    return second      # -inf means "no second max exists"
```

Dry run on `[3, 7, 7, 5]`:

```
x=3: first=3,  second=-inf
x=7: first=7,  second=3
x=7: nothing (7 is not > first, not < first)  <- duplicates handled by
STRICT comparisons
x=5: first=7,  second=5
```

Two classic mistakes: initializing `second = a[0]` (breaks when `a[0]` is the max), and using `>=` (then a duplicate of the max becomes "second max" — usually wrong). Second min is the same code with comparisons flipped.

3. Sorting — the patterns matter more than the algorithms

What is sorting? Arranging elements in order. Sorting only requires one thing: **a way to compare two elements**. That is why you can sort numbers, strings, or "students by marks, then by name" — you just supply the comparison.

You will rarely be asked "implement merge sort" in an interview. But each classic algorithm contains a **reusable pattern** that appears in dozens of real problems. That is why you must be able to write all of them from memory.

3.1 Selection sort — pattern: "repeatedly pick the extreme"

python

```
def selection_sort(a):
    n = len(a)
    for i in range(n):
        m = i
        for j in range(i + 1, n):
            if a[j] < a[m]:
                m = j
        a[i], a[m] = a[m], a[i]
```

$O(n^2)$. The pattern — *repeatedly extract the minimum* — is what a **heap** does efficiently; that's the seed of heapsort and of "top-K" problems.

3.2 Insertion sort — pattern: "grow a sorted prefix, shift to insert"

python

```
def insertion_sort(a):
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]          # shift right to make room
            j -= 1
        a[j + 1] = key
```

$O(n^2)$ worst case, but **$O(n)$ when nearly sorted** — interviewers love this fact. The pattern — *maintain a sorted region, shift elements to insert a new one* — appears in: inserting into a sorted list, online median-ish problems, and every "shift right to make room" move (compare with yesterday's `shift_left`!).

3.3 Merge sort — pattern: "divide and conquer + merge two sorted"

python

```

def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(left, right):
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
    return out

```

$O(n \log n)$ always, stable, needs $O(n)$ extra space. Two patterns inside it:

- **Divide and conquer** — split, solve halves, combine. Appears in: counting inversions, closest pair, "merge k lists", many tree problems.
- **Merge two sorted sequences** — the two-finger walk. Appears everywhere, including LeetCode 88 (Merge Sorted Array — which we solved *backward*, remember!).

3.4 Quick sort — pattern: "partition around a pivot"

```

python

def quick_sort(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo < hi:
        p = partition(a, lo, hi)
        quick_sort(a, lo, p - 1)
        quick_sort(a, p + 1, hi)

def partition(a, lo, hi):
    # Lomuto: pivot = a[hi]
    pivot = a[hi]
    i = lo
    # everything before i is < pivot
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    # pivot lands at its FINAL sorted position
    return i

```

$O(n \log n)$ average, $O(n^2)$ worst case, in place. The gold inside is **partition**: after one $O(n)$ pass, the pivot is in its final position, smaller elements left, bigger right. That single idea gives you:

- **Quickselect** — find the k -th smallest element in $O(n)$ average, *without sorting*:

```
python
def quickselect(a, k, lo=0, hi=None):    # k-th smallest, 0-indexed
    if hi is None:
        hi = len(a) - 1
    p = partition(a, lo, hi)
    if p == k: return a[p]
    if p < k: return quickselect(a, k, p + 1, hi)
    return quickselect(a, k, lo, p - 1)
```

- Dutch national flag / "sort colors", "move all negatives left", etc. — all are partitioning in disguise.

Summary table

Algorithm	Time (avg)	Time (worst)	Space	Stable	The reusable pattern
Selection	$O(n^2)$	$O(n^2)$	$O(1)$	no	repeatedly pick extreme ($\rightarrow$ heaps)
Insertion	$O(n^2)$	$O(n^2)$, $O(n)$ if nearly sorted	$O(1)$	yes	sorted prefix + shift to insert
Merge	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes	divide & conquer; merge two sorted
Quick	$O(n \log n)$	$O(n^2)$	$O(\log n)$	no	partition ($\rightarrow$ quickselect)

4. Binary search — TWO templates, not one

Binary search has two different jobs, and mixing up their templates causes infinite loops and off-by-one bugs. Learn them as two separate tools.

Pattern A: find an EXACT element

Search space: indices `[0 .. n-1]`. Loop while `[lo <= hi]`.

```
python
```

```
def find_exact(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Every step eliminates `mid` itself (that's why `mid + 1` / `mid - 1`), so the range strictly shrinks — no infinite loop possible.

Pattern B: find a BOUNDARY

Picture the array answering a yes/no question, and the answers look like `N N N N Y Y Y` — we want the **first Y**. (If there is no Y, the answer is `n`, "one past the end" — that is why the search range is `0 .. n`, one bigger!)

```
python

def first_true(n, ok):          # smallest i in [0, n] with ok(i) True; n if
    lo, hi = 0, n
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid             # mid could BE the answer -> keep it in range
        else:
            lo = mid + 1         # mid is a 'N' -> answer is strictly after it
    return lo
```

The golden rule: never write `lo = mid` in this template. Why: when `hi == lo + 1`, `mid = (lo+hi)//2 == lo`. If you then set `lo = mid`, nothing changes → **infinite loop**. `hi = mid` is safe (`mid < hi` always) and `lo = mid + 1` is safe (strictly increases). Check this every time you write it.

Differences at a glance:

	Exact	Boundary
range	0..n-1	0..n
loop	<code>lo <= hi</code>	<code>lo < hi</code>
moves	<code>lo = mid+1</code> , <code>hi = mid-1</code>	<code>lo = mid+1</code> , <code>hi = mid</code>
returns	index or -1	<code>lo</code> (<code>== hi</code>)

5. lower_bound, upper_bound, and the query table

Both are just Pattern B with a specific question:

```
python
def lower_bound(a, x):          # first index with a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] >= x:
            hi = mid
        else:
            lo = mid + 1
    return lo

def upper_bound(a, x):          # first index with a[i] > x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] > x:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

The ONLY difference: `>=` vs `>`. Everything else is the template.

Worked example: `a = [10, 20, 20, 20, 30, 40]` (indices 0..5)

```
lower_bound(a, 20) = 1    first element >= 20
upper_bound(a, 20) = 4    first element > 20
lower_bound(a, 25) = 4    (no 25: both point to the same place, index of 30)
upper_bound(a, 25) = 4
lower_bound(a, 50) = 6    nothing >= 50 -> "one past the end"
```

The query table — memorize this

Every "find something near x in a sorted array" question is one of these:

Query	Formula
Lower Bound	first index with value $\geq x$
Upper Bound	first index with value $> x$
Ceil of x (smallest value $\geq x$)	$a[\text{lowerBound}(x)]$
Floor of x (largest value $\leq x$)	$a[\text{upperBound}(x) - 1]$
First occurrence of x	$\text{lowerBound}(x)$ — then verify $a[i] == x$
Last occurrence of x	$\text{upperBound}(x) - 1$ — then verify $a[i] == x$
Smallest element $> x$	$a[\text{upperBound}(x)]$
Smallest element $\geq x$	$a[\text{lowerBound}(x)]$
Largest element $< x$	$a[\text{lowerBound}(x) - 1]$
Largest element $\leq x$	$a[\text{upperBound}(x) - 1]$
Insert position for x (keep sorted)	$\text{lowerBound}(x)$
Count of x	$\text{upperBound}(x) - \text{lowerBound}(x)$

Always check the edges: the formula may give (-1) or (n) — that means "no such element". And "verify equality" matters: $\text{lower_bound}(a, 25) = 4$, but $a[4] = 30 \neq 25$, so 25 has no first occurrence.

Check them all against $a = [10, 20, 20, 20, 30, 40]$, $x = 20$: $\text{ceil} = 20$, $\text{floor} = 20$, $\text{first} = 1$, $\text{last} = 3$, $\text{smallest} > x = 30$, $\text{largest} < x = 10$, $\text{count} = 3$. With $x = 25$: $\text{ceil} = 30$, $\text{floor} = 20$, $\text{first/last} = \text{none}$ (verify fails), $\text{insert position} = 4$.

6. Problem: find a repeated element

Array of size n contains elements in range $1..n-1$. Find any one repeated element.

First, why must a repeat exist? **Pigeonhole principle**: n values, only $n-1$ possible distinct numbers $\rightarrow$ at least two are equal.

We built a ladder of solutions, adding constraints each time:

Solution 1 — sort and scan adjacent

```
python
```

```
def find_duplicate_sort(a):
    a.sort()
    for i in range(1, len(a)):
        if a[i] == a[i - 1]:
            return a[i]
```

$O(n \log n)$ time, $O(1)$ extra — but it **modifies the array**.

Solution 2 — hash set

```
python

def find_duplicate_set(a):
    seen = set()
    for x in a:
        if x in seen:
            return x
    seen.add(x)
```

$O(n)$ time — but $O(n)$ extra space.

Constraint: array is IMMUTABLE and no extra space allowed

Solution 3 — binary search on the VALUE range — $O(n \log n)$, $O(1)$, no mutation

We binary search on **values** $1 \dots n-1$, not on indices! Question for a value mid : *how many array elements are $\leq \text{mid}$?* If that count is **more than mid**, then by pigeonhole the numbers $1 \dots \text{mid}$ are overcrowded $\rightarrow$ a duplicate lives in $[\text{lo}, \text{mid}]$. Otherwise it lives in $[\text{mid}+1, \text{hi}]$.

```
python

def find_duplicate_binsearch(a):
    lo, hi = 1, len(a) - 1                # search VALUES, not indices
    while lo < hi:
        mid = (lo + hi) // 2
        count = sum(1 for v in a if v <= mid)
        if count > mid:
            hi = mid                        # duplicate is in [lo, mid]
        else:
            lo = mid + 1                    # duplicate is in [mid+1, hi]
    return lo
```

Recognize the shape? **It is Pattern B (boundary template) applied to the value space.**

$\text{ok}(\text{mid}) = \text{count}(\leq \text{mid}) > \text{mid}$ looks like N N N Y Y Y over values, and we return the first Y. Same rules: $\text{hi} = \text{mid}$, never $\text{lo} = \text{mid}$.

Dry run on $a = [1, 3, 4, 2, 2]$ ($n = 5$, values in $1..4$):

```

lo=1, hi=4: mid=2, count(<=2) = |{1,2,2}| = 3 > 2 -> hi=2
lo=1, hi=2: mid=1, count(<=1) = |{1}| = 1 <= 1 -> lo=2
lo=hi=2 -> answer 2 ✓

```

Each count is one $O(n)$ scan, and we do $O(\log n)$ of them $\rightarrow O(n \log n)$ time, $O(1)$ space, array untouched.

Solution 4 — use the array itself as a hash map (negative marking)

If we ARE allowed to modify the array, we can do $O(n)$ time with no extra space. Key insight: values are $(1..n-1)$, and valid indices are $(0..n-1)$ — **every value is also a legal index**. So "have I seen value v ?" can be recorded by flipping the sign of $a[v]$:

```

python
def find_duplicate_marking(a):
    for i in range(len(a)):
        v = abs(a[i])          # abs(): the value may already be marked
        if a[v] < 0:
            return v          # visited twice -> v is repeated
        a[v] = -a[v]         # first visit: mark

```

Dry run on $a = [1, 3, 4, 2, 2]$:

```

i=0: v=|1|=1, a[1]=3 > 0 -> mark: [1, -3, 4, 2, 2]
i=1: v=|-3|=3, a[3]=2 > 0 -> mark: [1, -3, 4, -2, 2]
i=2: v=|4|=4, a[4]=2 > 0 -> mark: [1, -3, 4, -2, -2]
i=3: v=|-2|=2, a[2]=4 > 0 -> mark: [1, -3, -4, -2, -2]
i=4: v=|-2|=2, a[2]=-4 < 0 -> 2 already seen -> return 2 ✓

```

Two details that make it work: $\text{abs}(a[i])$ — because earlier iterations may have made $a[i]$ negative, but the *value stored there* is still $|a[i]|$; and the sign of $a[v]$ is spare, unused information (all inputs are positive) — we borrow it as a "seen" flag. You can restore the array afterward with one $\text{abs}()$ pass.

Compare the four

Solution	Time	Extra space	Mutates?
Sort + scan	$O(n \log n)$	$O(1)$	yes
Hash set	$O(n)$	$O(n)$	no
Binary search on values	$O(n \log n)$	$O(1)$	no
Negative marking	$O(n)$	$O(1)$	yes (restorable)

Notice how each new constraint killed a solution and forced a better idea. That is exactly how interviews go: "great — now do it without extra space."

(Further reading: LeetCode 287 adds "you may not modify the array AND must be $O(1)$ space AND better than $O(n \log n)$ " — the answer uses Floyd's cycle detection. Look it up after you are comfortable with the above.)

7. Run-length encoding, revisited

Recap of the algorithm (full code will be shared separately):

1. find_start -> a safe run 'pos' to begin from (the gas-station greedy on benefits E)
2. encode all runs, one sweep, starting at pos; the write cursor may WRAP past the array's end and continue at index 0
3. repair the layout: shift_left to close any hole, rotate_left to fix the order

After step 2 the answer is complete but sitting in the wrong place. There are exactly **two possible layouts**, and today we traced both.

Case 1 — TWO parts (writes did not wrap)

Example: `abTTTTTTTTTT` (n = 12)

```
runs: a(-1)  b(-1)  T×10 (E = 10-3 = +7)      find_start -> pos = 2

idx:  0 1 2 3 4 5 6 7 8 9 0 1
      a b T T T T T T T T T T      input
      a b T 1 0 T T T T T T T      encode from pos: T×10 -> 'T10' at [2..4]
      a b T 1 0 a 1 T T T T T      then the skipped runs: a1 at [5..6]
      a b T 1 0 a 1 b 1 T T T      b1 at [7..8] — cursor stopped at 9 <= n:

NO wrap

layout:  [junk][ T10 | a1 b1 ][junk]      <- answer in ONE piece, two halves:
          0..1  suffix prefix  9..11      suffix-tokens then prefix-
tokens

repair:  shift_left by 2   ->  T 1 0 a 1 b 1 . . . . .
          rotate_left by 3 ->  a 1 b 1 T 1 0 . . . . .   (3 = size of
'T10')

answer = a1b1T10
```

Case 2 — THREE parts with a hole (writes wrapped)

Example: `abcddeefffg` (n = 15)

```

runs: a(-1) b(-1) c(-1) d×6(+4) ee(0) fff(+1) g(-1)    find_start -> pos = 3

idx:  0 1 2 3 4 5 6 7 8 9 0 1 2 3 4
      a b c d d d d d e e f f f g    input
      a b c d 6 e 2 f 3 g 1 f f f g    encode from pos: d6 e2 f3 g1
      a b c d 6 e 2 f 3 g 1 a 1 b 1    skipped runs: a1, b1 exactly fill the
tail
      c 1 c d 6 e 2 f 3 g 1 a 1 b 1    c1 WRAPS to the front!

layout:  [ c1 ][hole][ d6 e2 f3 g1 ][ a1 b1 ]
          part3  idx2      part2      part1      <- answer in THREE parts:
                                                    prefix(a1 b1) is split
from
                                                    the rest by physical
position!

The cell at index 2 still holds a stale 'c'. It LOOKS like data – it is junk.
We know that by POSITION (it lies in the hole), never by content.

repair:  shift_left by 1 (close the hole):
          c 1 d 6 e 2 f 3 g 1 a 1 b 1 .
          rotate_left by 10 (= hole-end 2 + suffix size 8):
          a 1 b 1 c 1 d 6 e 2 f 3 g 1 .

answer = a1b1c1d6e2f3g1

```

Case 2 again, with a wider hole: **abccdddefggggghhhh** (n = 19)

```

runs: a(-1) b(-1) cc(0) ddd(+1) e(-1) f(-1) g×5(+3) h×5(+3)    pos = 9

      a b c c d d d e f g g g g g h h h h h    input
      a b c c d d d e f g 5 h 5 a 1 b 1 c 2    encode g5 h5, then a1 b1 c2
fill tail
      d 3 e 1 f 1 d e f g 5 h 5 a 1 b 1 c 2    d3 e1 f1 wrap to the front

layout:  [ d3 e1 f1 ][ d e f ][ g5 h5 ][ a1 b1 c2 ]
          wrapped    HOLE    suffix    prefix
          [0..5]     [6..8]   [9..12]   [13..18]

repair:  shift_left by 3 -> d3 e1 f1 g5 h5 a1 b1 c2 (junk...)
          rotate_left by 10 -> a1 b1 c2 d3 e1 f1 g5 h5

answer = a1b1c2d3e1f1g5h5

```

The rule connecting both cases: after encoding, ask "**did the write cursor pass the end of the array?**" No → one piece, just slide it to the front and rotate. Yes → two pieces with a hole between them: close the hole (shift), then rotate. The rotation exists in both cases for the same reason — we started encoding from the middle run, so the tokens of the string's beginning come after the tokens of its end.

8. The real lesson: small modules

Look at the complete solution's ingredient list:

<code>reverse(arr, lo, hi)</code>	4 lines
<code>rotate_left(arr, len, k)</code>	3 reverses
<code>shift_left(arr, lo, hi, gap)</code>	2 lines
<code>run_end(arr, i, hi)</code>	3 lines
<code>token_length(run)</code>	1 line
<code>find_start(arr)</code>	~10 lines (gas-station greedy)
<code>write_token / encode_range</code>	~10 lines

<code>compress(arr)</code>	~12 lines of ORCHESTRATION

Not one of these functions is hard. Each one you can write, test, and trust **independently** — we tested `reverse` before `rotate_left` existed, and `rotate_left` before `compress` existed. By the time we assembled `compress`, every brick was already solid, so the only thing that could be wrong was the assembly — and that is exactly where we looked when debugging.

Compare with writing one 80-line `compress()` directly: any bug could be anywhere, nothing can be tested alone, and the code cannot be read aloud.

This is the habit to build:

Break the problem into functions so small they are obviously correct. Test each one alone. Then the big problem becomes a short story told with words you already trust.

The same lesson appeared everywhere today: `partition` inside quicksort, `merge` inside merge sort, `ok(mid)` inside the boundary template, `lower_bound`/`upper_bound` powering the entire query table. Big algorithms are small functions glued together — learn the small functions.